

Tecnológico de Costa Rica

Principios de Sistemas Operativos [IC-6600]

Sede: Cartago

Profesor: Armando Arce Orozco

Proyecto 3: Sistema de archivos S3

Integrantes:

Fernando Andrés Sánchez Hidalgo [2022218688]

Kevin Espinoza Solano [2023055841]

I Semestre 2026

Repositorio:

https://github.com/fash2110/OS-3_2026

Índice

1	Introducción	2
2	Descripción del problema	2
3	Definición de estructuras de datos	4
3.1	Structs	4
3.1.1	Servidor	4
3.1.2	Cliente	5
3.2	Variables globales	6
4	Componentes principales	7
5	Mecanismo de creación de archivos y comunicación con el servidor	13
5.1	Estructura del archivo .bkt	13
5.2	Algoritmo de asignación de espacio (first-fit)	13
5.3	Protocolo de comunicación cliente-servidor	13
6	Pruebas	14
6.1	Metodología	14
6.2	Resultados	15
7	Conclusiones	16

1. Introducción

El proyecto tiene como objetivo simular un sistema de archivos basado en buckets similar al S3 (Simple Storage Service) de AWS. Para esto se implementan dos programas, un servidor que maneja operaciones de buckets y sus objetos (Creación, Modificación, Obtención y Eliminación) y recibe comandos del programa cliente. El programa cliente obtiene comandos del usuario por medio de la consola y los traslada para que el servidor realice las operaciones del sistema de archivos.

2. Descripción del problema

Para la implementación del proyecto, se generan dos programas escritos en C

- Servidor: Enviar/recibir mensajes al/del cliente y realizar operaciones en sistema de archivos
- Cliente: Interfaz del usuario, enviar y recibir mensajes al servidor

La estrategia del programa servidor:

1. Iniciar el servidor TCP en el puerto 8080 y quedar a la espera de conexiones entrantes mediante `accept()`.
2. Al conectarse un cliente, recibir la cabecera fija del mensaje (`MsgHeader`) que indica el tipo de comando, flags y longitudes de los campos.
3. Leer secuencialmente del socket: el nombre del bucket, la clave del objeto (*key*), una segunda clave opcional (*key2*) y los datos binarios adjuntos, si los hay.
4. Determinar el comando solicitado y despacharlo al manejador correspondiente:
 - `CMD_LS`: listar buckets o el contenido de un bucket filtrando por prefijo.
 - `CMD_MB`: crear un nuevo archivo `.bkt` que representa el bucket.
 - `CMD_PUT`: almacenar un objeto en el bucket, reutilizando espacio libre si existe o escribiendo al final del archivo.
 - `CMD_GET`: recuperar el contenido de un objeto del bucket y enviarlo al cliente.
 - `CMD_MV`: copiar el objeto al destino y eliminar el original.
 - `CMD_RM`: eliminar uno o varios objetos, opcionalmente de forma recursiva por prefijo.

- `CMD_SYNC`: confirmar disponibilidad para recibir múltiples objetos vía `CMD_PUT`.
 - `CMD_RB`: eliminar un bucket, con verificación de que esté vacío a menos que se use el flag `-force`.
5. Enviar al cliente una respuesta con cabecera `MsgHeader`, indicando `RESP_OK`, `RESP_ERROR`, `RESP_DATA` o `RESP_LIST` según corresponda, junto con datos adjuntos si la operación lo requiere.
 6. Regresar al paso 2 para atender el siguiente comando del mismo cliente, hasta que este cierre la conexión.

Estrategia del programa cliente:

1. Recibir el comando del usuario por línea de argumentos (`argc`, `argv`) e identificar la operación solicitada: `ls`, `mb`, `cp`, `mv`, `rm`, `sync` o `rb`.
2. Analizar los argumentos restantes, parsear las URIs con formato `s3://bucket/key` para extraer el nombre del bucket y la clave del objeto, e identificar las banderas opcionales (`-recursive`, `-delete`, `-force`).
3. Establecer una conexión TCP con el servidor en `127.0.0.1:8080` mediante `connect_to_server()`.
4. Construir y enviar la cabecera `msgHeader` seguida del nombre del bucket, la clave del objeto, una segunda clave opcional (`key2`) y los datos binarios adjuntos si la operación lo requiere (por ejemplo, `cp local → S3`).
5. Recibir la respuesta del servidor mediante `recv_response()`, que retorna el tipo de respuesta (`RESP_OK`, `RESP_ERROR`, `RESP_DATA`, `RESP_LIST`) junto con los datos adjuntos si los hay.
6. Procesar la respuesta: mostrar el resultado al usuario, escribir archivos locales en caso de descarga, o reportar el error correspondiente.
7. Cerrar la conexión con el servidor. Para operaciones que requieren múltiples intercambios (como `sync` o `cp -recursive`), se establece una nueva conexión por cada operación individual.

3. Definición de estructuras de datos

3.1. Structs

3.1.1. Servidor

```
typedef struct __attribute__((packed)) {
    uint8_t  cmd;           /* tipo de comando / respuesta */
    uint8_t  flags;         /* bits auxiliares (--recursive=1, --
                             force=2, --delete=4) */
    uint16_t bucket_len;    /* longitud del nombre del bucket (
                             sin '\0') */
    uint16_t key_len;       /* longitud de la clave/prefijo (
                             sin '\0') */
    uint16_t key2_len;      /* clave destino en mv (
                             sin '\0') */
    uint64_t data_len;      /* longitud de los datos adjuntos */
} msgHeader;
```

Cabecera que precede a todos los mensajes recibidos o emitidos por el servidor, contiene información como el tipo de comando enviado, banderas auxiliares, longitud del identificador del bucket, longitud del nombre de los objetos incluidos y longitud de los datos.

```
typedef struct {
    char      magic[4];      // firma "BKT1" para archivos
                          binarios
    uint32_t  max_entries;
    uint32_t  num_entries;   // entradas de directorio ocupadas
    uint32_t  max_free;
    uint32_t  num_free;      // entradas de free-list ocupadas
    uint64_t  data_start;    // offset donde empieza la zona de
                          datos
} bucketHeader;
```

Cabecera de cada bucket para gestión de sus objetos correspondientes. Todo bucket empieza con la firma BKT1 por ser un archivo binario. En la cabecera se mantiene información como la capacidad máxima del bucket, cantidad de entradas ocupadas, capacidad máxima de espacios libres, cantidad de espacios libres actuales y un offset del inicio del espacio de datos calculado con `data_start=sizeof(BucketHeader) +`

$\text{MAX_ENTRIES} \times \text{sizeof}(\text{DirEntry}) + \text{MAX_FREE} \times \text{sizeof}(\text{FreeBlock})$ para garantizar que los metadatos y contenido del bucket no contengan intersección.

```
typedef struct {
    char      key[MAX_KEY_LEN];        // clave/prefijo completo,
    " " si está libre
    uint64_t  offset;                  // byte donde empieza el
    contenido
    uint64_t  size;                    // tamaño en bytes del
    objeto
    int       used;                    // 1 = slot ocupado, 0 =
    libre
} dirEntry;
```

Entrada de directorio de un objeto en el bucket para búsqueda directa del contenido, se tiene un campo para el identificador del objeto (ej: `fotos/vacaciones/2024.jpg` o si es un espacio libre), un offset que representa la distancia desde el inicio del archivo hasta el primer byte del contenido para búsqueda con `fseek()`, el tamaño del contenido del objeto y un bit para definir si es un campo libre u ocupado.

```
typedef struct {
    uint64_t  offset;
    uint64_t  size;
    int       used;    // 1 = hueco libre válido, 0 = slot de la
    lista sin usar
} freeBlock;
```

Espacio libre en el contenido del bucket, contiene el offset para su búsqueda directa en la zona de datos, tamaño del espacio libre para identificar si es lo suficientemente grande para el contenido que lo va a reemplazar y un bit que define si es un espacio válido u ocupado.

3.1.2. Cliente

```
typedef struct __attribute__((packed)) {
    uint8_t  cmd;
    uint8_t  flags;
    uint16_t bucket_len;
```

```
    uint16_t key_len;  
    uint16_t key2_len;  
    uint64_t data_len;  
} msgHeader;
```

La estructura `msgHeader` del cliente es idéntica a la del servidor y debe mantenerse así por diseño: ambos extremos de la comunicación deben interpretar los mismos bytes de la misma forma. El cliente la utiliza exclusivamente para construir y enviar mensajes, mientras que el servidor la utiliza para recibirlos e interpretarlos.

3.2. Variables globales

Para ambos programas de cliente y servidor, se utilizan las mismas variables globales, ya que se requiere la misma configuración para una comunicación exitosa.

- `MAX_KEY_LEN`: Tamaño máximo para nombre de un object
- `MAX_ENTRIES`: Capacidad máxima de un bucket
- `MAX_FREE`: Capacidad máxima del espacio libre
- `BUCKETS_DIR`: Directorio donde se almacenan los buckets del lado del servidor (`./buckets`)
- `SERVER_PORT`: Puerto para recibir mensajes (8080)
- `BACKLOG`: Cantidad de clientes que permite el servidor para operaciones secuenciales (**10**)
- enum de comandos de mensajes
 - `CMD_LS`: 1
 - `CMD_MB`: 2
 - `CMD_PUT`: 3
 - `CMD_GET`: 4
 - `CMD_MV`: 5
 - `CMD_RM`: 6
 - `CMD_SYNC`: 7
 - `CMD_RB`: 8

- Códigos de respuesta
 - RESP_OK: 100
 - RESP_ERROR: 101
 - RESP_DATA: 102
 - RESP_LIST: 103
- Banderas auxiliares
 - FLAG_RECURSIVE: 0x01
 - FLAG_FORCE: 0x02
 - FLAG_DELETE: 0x04

4. Componentes principales

Servidor

- `main(void) → int`
 - Inicializa el socket TCP en el puerto 8080
 - Queda en espera de conexiones entrantes y delega cada cliente a `handle_client()`
- `handle_client(int client_fd) → void`
 - Componente central del servidor
 - Recibe la cabecera `MsgHeader` y los campos del mensaje (`bucket`, `key`, `key2`, `datos`)
 - Despacha el comando al manejador correspondiente según el campo `cmd`
 - Mantiene el ciclo de atención hasta que el cliente cierra la conexión

Manejadores de comandos

- `handle_ls(int fd, char* bucket, char* prefix) → void`
 - Lista todos los buckets disponibles si `bucket` está vacío
 - Lista los objetos de un bucket filtrando por prefijo

- `handle_mb(int fd, char* bucket) → void`
 - Crea un nuevo archivo `.bkt` con su cabecera, directorio y lista de espacios libres inicializados
- `handle_put(int fd, char* bucket, char* key, unsigned char* data, uint64_t data_len) → void`
 - Almacena un objeto en el bucket
 - Reutiliza un `FreeBlock` disponible mediante *first-fit*, o escribe al final del archivo
 - Crea el bucket automáticamente si no existe
- `handle_get(int fd, char* bucket, char* key) → void`
 - Recupera el contenido de un objeto del bucket y lo envía al cliente como `RESP_DATA`
- `handle_mv(int fd, char* bucket_src, char* key_src, char* bucket_dst, char* key_dst) → void`
 - Copia el objeto al bucket destino y elimina el original
 - Crea el bucket destino automáticamente si no existe
- `handle_rm(int fd, char* bucket, char* key, int recursive) → void`
 - Elimina un objeto específico o, en modo recursivo, todos los objetos cuya clave inicie con el prefijo dado
- `handle_rb(int fd, char* bucket, int force) → void`
 - Elimina un bucket verificando que esté vacío
 - Con `force=1` elimina el bucket aunque contenga objetos

Gestión de buckets

- `bucket_create(char* bucket_name) → int`
 - Crea el archivo `.bkt` e inicializa el `BucketHeader`, el arreglo de `DirEntry` y la lista de `FreeBlock`

- `bucket_put_object(char* bucket, char* key, unsigned char* data, uint64_t size) → int`
 - Busca si el objeto ya existe para reemplazarlo
 - Aplica el algoritmo *first-fit* sobre la lista de **FreeBlock** para asignar espacio
 - Actualiza la **DirEntry** correspondiente y la lista de espacios libres
- `bucket_get_object(char* bucket, char* key, uint64_t* out_size) → unsigned char*`
 - Localiza el objeto por su clave en el directorio
 - Retorna un buffer con el contenido; el llamador es responsable de liberar la memoria con `free()`
- `bucket_delete_object(char* bucket, char* key) → int`
 - Marca la **DirEntry** como libre (`used=0`)
 - Registra el espacio liberado como un nuevo **FreeBlock**
- `bucket_remove(char* bucket, int force) → int`
 - Verifica si el bucket contiene objetos antes de eliminarlo
 - Elimina el archivo `.bkt` del sistema de archivos

Helpers de red

- `net_send_all(int fd, void* buf, size_t len) → int`
 - Garantiza el envío completo de `len` bytes al socket, reintentando si `send()` transmite menos bytes de los esperados
- `net_recv_all(int fd, void* buf, size_t len) → int`
 - Garantiza la recepción completa de `len` bytes del socket, reintentando si `recv()` entrega menos bytes de los esperados
- `send_response(int fd, uint8_t type, char* msg) → void`
 - Envía una respuesta simple al cliente con cabecera **MsgHeader** y un mensaje de texto opcional

- `send_data_response(int fd, uint8_t type, void* data, uint64_t len) → void`
 - Envía una respuesta con datos binarios adjuntos, utilizada en operaciones GET y LS

Cliente

- `main(int argc, char* argv[]) → int`
 - Identifica el comando solicitado comparando `argv[1]` contra los comandos disponibles
 - Desplaza `argv` para que cada subcomando reciba sus argumentos desde `argv[0]`
 - Delega la ejecución al comando correspondiente e imprime el uso correcto si el comando es desconocido

Comandos

- `cmd_ls(int argc, char* argv[]) → int`
 - Si se provee una URI `s3://bucket[/prefijo]`, lista los objetos del bucket filtrando por prefijo
 - Sin argumentos, solicita al servidor la lista de todos los buckets disponibles
- `cmd_mb(int argc, char* argv[]) → int`
 - Parsea la URI `s3://bucket` y envía `CMD_MB` al servidor para crear el bucket
- `cmd_cp(int argc, char* argv[]) → int`
 - Determina la dirección de la copia según si los argumentos son URIs S3 o rutas locales:
 - Local → S3: lee el archivo con `read_file()` y lo envía con `do_put()`. Con `-recursive` recorre el directorio con `opendir()` y sube cada archivo manteniendo la ruta relativa como clave
 - S3 → local: descarga el objeto con `do_get()`. Con `-recursive` obtiene primero el listado del bucket y descarga cada objeto individualmente

- S3 → S3: descarga el objeto origen con `CMD_GET` y lo sube al destino con `CMD_PUT` en una nueva conexión
- `cmd_mv(int argc, char* argv[]) → int`
 - Local → S3: reutiliza la lógica de `cmd_cp` y elimina el archivo local con `remove()`
 - S3 → local: descarga el objeto con `do_get()` y lo elimina del bucket con `CMD_RM`
 - S3 → S3: envía `CMD_MV` con `key2 = "bucket_dst\nkey_dst"` para que el servidor realice la operación atómicamente
- `cmd_rm(int argc, char* argv[]) → int`
 - Envía `CMD_RM` con la bandera `FLAG_RECURSIVE` activa si se especifica `-recursive`, eliminando todos los objetos cuya clave inicie con el prefijo dado
- `cmd_sync(int argc, char* argv[]) → int`
 - Local → S3: recorre el directorio local y sube cada archivo con `do_put()`. Con `-delete` obtiene el listado del bucket y elimina los objetos remotos que ya no existen localmente
 - S3 → local: obtiene el listado del bucket y descarga únicamente los objetos cuyo tamaño difiere del archivo local correspondiente, omitiendo los que ya están sincronizados
- `cmd_rb(int argc, char* argv[]) → int`
 - Envía `CMD_RB` con la bandera `FLAG_FORCE` activa si se especifica `-force`, permitiendo eliminar buckets que aún contengan objetos

Helpers de red y sistema de archivos

- `connect_to_server(void) → int`
 - Crea un socket TCP y establece la conexión con el servidor en `SERVER_HOST:SERVER_PORT`
 - Retorna el file descriptor del socket o `-1` en caso de error
- `send_header(int fd, uint8_t cmd, uint8_t flags, char* bucket, char* key, char* key2, uint64_t data_len) → int`

- Construye y envía la cabecera `msgHeader` seguida de los campos `bucket`, `key` y `key2`, calculando automáticamente las longitudes de cada campo
- `recv_response(int fd, unsigned char** out_data, uint64_t* out_len) → int`
 - Lee la cabecera de respuesta del servidor y, si `data_len > 0`, recibe los datos adjuntos en un buffer asignado dinámicamente
 - El llamador es responsable de liberar el buffer con `free()`
- `do_put(int fd, char* local, char* bucket, char* key) → int`
 - Lee el archivo local con `read_file()`, envía la cabecera `CMD_PUT` seguida del contenido binario y verifica la respuesta del servidor
- `do_get(int fd, char* bucket, char* key, char* local) → int`
 - Envía `CMD_GET` y escribe el contenido recibido en la ruta local con `write_file()`
 - Si `local` es un directorio, construye el nombre del archivo usando la parte final de la clave
- `parse_s3_uri(char* uri, char* bucket, char* key) → int`
 - Divide una URI `s3://bucket/key` en sus componentes `bucket` y `key`
 - Retorna 0 si la URI es válida o -1 si no tiene el prefijo `s3://`
- `read_file(char* path, uint64_t* out_size) → unsigned char*`
 - Lee el contenido completo de un archivo local en un buffer dinámico; el llamador debe liberar la memoria con `free()`
- `write_file(char* path, unsigned char* data, uint64_t size) → int`
 - Escribe datos en una ruta local, creando los directorios intermedios necesarios con `mkdir()`

5. Mecanismo de creación de archivos y comunicación con el servidor

5.1. Estructura del archivo `.bkt`

Cada bucket se representa como un único archivo binario con extensión `.bkt` almacenado en el directorio `./buckets/` del servidor. Al crear un bucket con `bucket_create()`, se escribe una estructura fija compuesta por tres zonas contiguas:

1. **Cabecera** (`bucketHeader`): ocupa los primeros bytes del archivo e incluye la firma `BKT1`, las capacidades máximas y el offset `data_start` que delimita el inicio de la zona de datos.
2. **Directorio de objetos**: arreglo de `MAX_ENTRIES` entradas `dirEntry`, cada una inicializada con `used=0`. Ocupa `MAX_ENTRIES × sizeof(dirEntry)` bytes inmediatamente después de la cabecera.
3. **Lista de espacios libres**: arreglo de `MAX_FREE` entradas `freeBlock`, cada una inicializada con `used=0`. Ocupa `MAX_FREE × sizeof(freeBlock)` bytes a continuación del directorio.
4. **Zona de datos**: región donde se escribe el contenido binario de los objetos, a partir del offset `data_start`.

5.2. Algoritmo de asignación de espacio (`first-fit`)

Cuando se almacena un objeto nuevo o se reemplaza uno existente con un tamaño diferente, la función `allocate_space()` recorre la lista de `freeBlock` en orden y selecciona el primer hueco cuyo campo `size` sea mayor o igual al tamaño requerido. Si no existe ningún hueco suficiente, el objeto se escribe al final del archivo, calculando el offset con `data_end_offset()`. Una vez utilizado un `freeBlock`, su campo `used` se marca como 0 para liberarlo de la lista. Cuando un objeto es eliminado o reemplazado, su espacio se registra como un nuevo `freeBlock` mediante `add_free_block()`.

5.3. Protocolo de comunicación cliente-servidor

La comunicación se realiza sobre una conexión TCP entre el cliente y el servidor en el puerto 8080. Todo intercambio de mensajes sigue el mismo formato:

1. El cliente envía una cabecera fija de 20 bytes (`msgHeader`) con el tipo de comando, banderas y las longitudes de los campos que siguen.
2. A continuación envía secuencialmente: el nombre del bucket, la clave del objeto (*key*), una segunda clave opcional (*key2*, usada en `CMD_MV`), y los datos binarios adjuntos si `data_len > 0`.
3. El servidor procesa el comando y responde con otra `msgHeader` indicando el resultado (`RESP_OK`, `RESP_ERROR`, `RESP_DATA` o `RESP_LIST`), seguida de los datos adjuntos si los hay.

Las funciones `net_send_all()` y `net_recv_all()` garantizan la transmisión completa de cada bloque de bytes, reintentando la llamada al sistema si `send()` o `recv()` transfieren menos bytes de los solicitados, lo cual es habitual en sockets TCP. Esto permite transmitir correctamente tanto texto como archivos binarios (imágenes, ejecutables, etc.).

6. Pruebas

6.1. Metodología

Las pruebas fueron realizadas desde un sistema windows utilizando la herramienta de WSL (Windows subsystem for Linux) utilizando la siguiente configuración en la figura 1. Primeramente se levanta el servidor utilizando

```
>> cd servidor
>> ./aws-s3_server
```

Una vez se tiene el servidor levantado, se pueden empezar a realizar las pruebas desde el directorio cliente

```
>> cd ..
>> cd cliente
>> cd pruebas
>> ./benchmark.sh
```

Se generó un script de pruebas que puede ser encontrado en el repositorio de github en la ubicación `/cliente/pruebas/benchmark.sh`, dentro del script de pruebas se encuentran las siguientes pruebas de rendimiento en la tabla 1

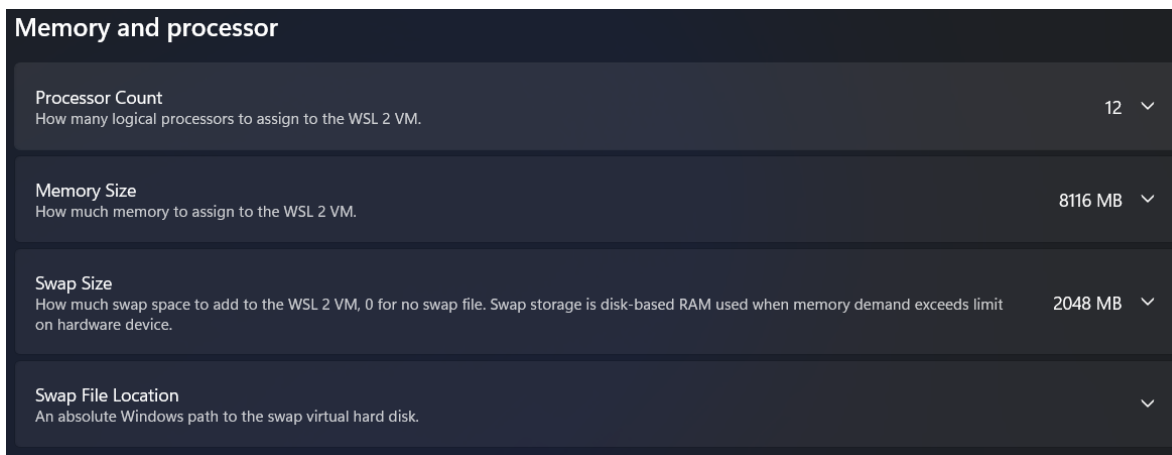


Figura 1: Configuración de la maquina virtual WSL

Cuadro 1: Pruebas de rendimiento	
Prueba	Descripción
latencia	crear, listar (con y sin bucket) y eliminar bucket
subida	subida de archivos (10KB, 512KB, 5MB, 20MB)
descarga	descarga de archivos (S3 → local)
sync	sincronización (local → S3)
move + remove	mover archivos dentro del bucket, eliminar objeto en el bucket, eliminar objetos recursivamente
sobreescritura	sobreescribir archivos con mismo y diferente tamaño

6.2. Resultados

Una vez ejecutado el script de pruebas, se genera un archivo `/cliente/pruebas/resultados_benchmark.txt` en el cual se documenta el resultado de las pruebas realizadas, los resultados se pueden visualizar en la tabla 2

Se puede visualizar en todas las pruebas que los tiempo de respuesta se centran entre 2 y 3ms, esto se debe en gran parte a que ambos programas cliente y servidor son ejecutados dentro del mismo sistema. En general, los resultados demuestran que la arquitectura de archivo único por bucket con directorio de entradas de tamaño fijo permite tiempos de acceso predecibles y constantes, sin cambios observables al aumentar el tamaño de los objetos, o la cantidad de archivos sincronizados en el entorno de prueba

Cuadro 2: Resultados de pruebas

Prueba	tiempo (ms)
latencia	
mb	2
ls (listar buckets)	3
ls (listar objetos)	2
rb (vacío)	3
subida	
10 KB	3
512KB	2
5MB	3
20MB	3
descarga	
10 KB	3
512KB	2
5MB	3
20MB	2
sync	
10 archivos	2
50 archivos	2
100 archivos	3
move + remove	
mv (mismo bucket)	2
rm (objeto)	3
rm -recursive	2
sobreescritura	
mismo tamaño	2
distinto tamaño	2

utilizado.

7. Conclusiones

- La representación de cada bucket como un único archivo binario `.bkt` con una zona de metadatos fija al inicio y una zona de datos variable al final demostró ser una estrategia eficaz para simular un sistema de almacenamiento de objetos, evitando la dependencia del sistema de archivos del sistema operativo para la organización interna de los objetos. Las pruebas de latencia base confirmaron esto, con operaciones de metadatos como `mb`, `ls` y `rb` completando consistentemente

en 2–3 ms.

- El uso del atributo `__attribute__((packed))` en la estructura `msgHeader` fue esencial para garantizar que la cabecera ocupe exactamente 20 bytes en ambos extremos de la comunicación, eliminando problemas de alineación de memoria que podrían corromper el protocolo entre cliente y servidor.
- El algoritmo *first-fit* implementado en `allocate_space()` permite reutilizar los espacios liberados por objetos eliminados o reemplazados, evitando el crecimiento indefinido del archivo `.bkt`. Las pruebas de sobreescritura confirmaron que tanto la sustitución de objetos del mismo tamaño como de distinto tamaño se realizan en 2 ms, sin diferencia medible entre ambos casos en entorno local. Sin embargo, el no dividir un bloque libre mayor en dos partes tras una asignación parcial puede generar fragmentación interna con el tiempo.
- Los tiempos de subida y descarga se mantuvieron estables entre 2 y 3 ms independientemente del tamaño del archivo transferido, desde 10 KB hasta 20 MB. Dado que las pruebas se ejecutaron con cliente y servidor en la misma máquina sobre el adaptador de loopback (127.0.0.1), el factor dominante fue el acceso a disco y no la latencia de red, lo que indica que el rendimiento en un entorno de red real podría variar significativamente.
- Las pruebas de `sync` mostraron tiempos consistentes de 2–3 ms al escalar de 10 a 100 archivos, lo que sugiere que el costo por conexión TCP en entorno local es despreciable. Sin embargo, dado que `cmd_sync()` establece una conexión independiente por cada archivo, en una red real con latencia apreciable este comportamiento podría convertirse en un cuello de botella, siendo una mejora natural el uso de conexiones persistentes o el envío en grupos.
- La separación clara entre el programa cliente y el servidor permite que el cliente sea una interfaz liviana que únicamente construye mensajes y presenta resultados, mientras que toda la lógica de gestión de archivos reside en el servidor, lo que facilita la extensión futura del sistema.
- El servidor atiende clientes de forma secuencial, por lo que operaciones como `sync` o `cp -recursive`, que requieren múltiples conexiones, son procesadas una a la vez. Una mejora natural sería introducir concurrencia mediante hilos o procesos hijos para atender múltiples clientes simultáneamente, especialmente consideran-

do que las pruebas se realizaron en un entorno de un único cliente y los tiempos podrían degradarse considerablemente bajo carga concurrente real.